

DOM manipulation

COMPLETE LEARNING GUIDE

What You'll Learn

The Document Object Model (DOM) is the browser's live, in-memory tree representation of a page, and vanilla DOM manipulation — selecting elements, changing their content and attributes, responding to events, and updating the tree — is what every frontend framework ultimately compiles down to. Understanding it directly matters even in a React/Vue/Angular-heavy career, both because you'll occasionally need to escape the framework abstraction (measuring an element, integrating a non-framework library) and because understanding what the framework is optimizing away makes framework-level performance problems make sense. This submodule covers selecting and creating elements, updating content and attributes safely, the event system including bubbling and delegation, and the handful of practices that keep DOM-heavy code fast instead of janky.

1. Selecting Elements

`document.querySelector(selector)` returns the first element matching any CSS selector, and `document.querySelectorAll(selector)` returns all matches as a static `NodeList` — both are the modern, flexible default. The older `getElementById`, `getElementsByClassName`, and `getElementsByTagName` are faster in theory (no CSS selector parsing) but far less flexible, and `getElementsByClassName/TagName` return a live `HTMLCollection` that automatically updates as the DOM changes, which can cause subtle bugs if you iterate over it while modifying the DOM. `querySelectorAll`'s `NodeList`, by contrast, is a snapshot and supports `.forEach()` directly.

JAVASCRIPT

```
const submitBtn = document.querySelector('#submit-btn');
const allCards = document.querySelectorAll('.card');

allCards.forEach((card) => {
  card.classList.add('loaded');
});

if (submitBtn === null) {
  console.warn('Submit button not found in DOM'); // always guard against null
}
```

2. Creating and Updating Elements Safely

`document.createElement(tag)` builds a new, detached element that must be inserted with `appendChild`, `append`, `prepend`, or `insertBefore` before it appears on the page. For updating existing content, `textContent` sets/reads plain text and is always safe; `innerHTML` parses a string as HTML and re-renders the element's children, which is powerful but dangerous — inserting any string that includes untrusted user input via `innerHTML` opens a cross-site scripting (XSS) vulnerability, since embedded `<script>` or event-handler attributes can execute. When you need to insert structured content, build it with `createElement/append` calls or use the safer `insertAdjacentText`, rather than string-concatenating HTML.

JAVASCRIPT

```
const list = document.querySelector('#todo-list');

function addTodo(text) {
  const li = document.createElement('li');
  li.textContent = text; // safe – never interprets `text` as HTML
  li.classList.add('todo-item');
  list.appendChild(li);
}

addTodo('<img src=x onerror=alert(1)>'); // rendered as literal text, not
executed
```

3. Event Handling and the Event Object

`element.addEventListener(type, handler)` attaches a listener without overwriting any existing listeners on the same element (unlike the older `element.onclick = handler`, which allows only one). The handler receives an event object carrying details like `event.target` (the actual element that triggered the event, useful with delegation), `event.currentTarget` (the element the listener is attached to), and methods like `event.preventDefault()` (stop a form submission or link navigation) and `event.stopPropagation()` (stop the event from bubbling further up the tree).

JAVASCRIPT

```
const form = document.querySelector('#signup-form');

form.addEventListener('submit', (event) => {
  event.preventDefault(); // stop the default full-page form submission
  const formData = new FormData(form);
  console.log(Object.fromEntries(formData));
});
```

4. Event Bubbling and Delegation

Most DOM events bubble: after firing on the exact target element, they fire again on each ancestor up to document. This is what makes event delegation possible — instead of attaching a listener to every individual item in a list (expensive, and broken for items added later), attach one listener to a stable parent container and inspect `event.target` to determine which child was actually interacted with. Delegation is both a performance win (one listener instead of hundreds) and a correctness win (it automatically covers elements added to the

DOM after the listener was attached).

JAVASCRIPT

```
const list = document.querySelector('#todo-list');

// One listener handles clicks on ANY current or future <li>, via delegation
list.addEventListener('click', (event) => {
  const item = event.target.closest('li');
  if (!item) return; // click didn't land on/inside an <li>
  item.classList.toggle('done');
});
```

5. Classes, Styles, and Data Attributes

`classList` (`.add`, `.remove`, `.toggle`, `.contains`) is the preferred way to change an element's appearance, since it works with existing CSS rules instead of hard-coding style values in JavaScript. The `style` property sets inline CSS directly (`element.style.display = 'none'`) and is appropriate for values computed at runtime that can't be expressed as a toggleable class, like a dynamically calculated position. `dataset` exposes custom `data-*` attributes as a convenient object, which is the standard way to attach small pieces of state or configuration directly to an element in HTML.

JAVASCRIPT

```
// <button data-user-id="42" class="follow-btn">Follow</button>
const btn = document.querySelector('.follow-btn');
console.log(btn.dataset.userId); // "42" (always a string)

btn.addEventListener('click', () => {
  btn.classList.toggle('following');
  btn.textContent = btn.classList.contains('following') ? 'Following' : 'Follow';
});
```

6. Traversing the DOM

Once you have a reference to one element, related elements are reachable via `parentElement`, `children` (element children only, unlike `childNodes` which includes text/comment nodes), `nextElementSibling`/`previousElementSibling`, and `closest(selector)` (walks up from the element, including itself, to find the nearest ancestor matching a selector — extremely useful inside delegated event handlers, as shown above).

7. Performance: Avoiding Layout Thrashing

Reading a layout property (`offsetHeight`, `getBoundingClientRect()`, `scrollTop`) forces the browser to recalculate layout if anything has changed since the last render; writing a style change invalidates layout again. Interleaving reads and writes in a loop — read, write, read, write — forces the browser to recalculate layout repeatedly within a single frame ('layout thrashing'), which is a common cause of janky scrolling and animation. The fix is to batch all reads first, then all writes. For inserting many elements at once, building them inside a `DocumentFragment` (an in-memory container with no visual presence) and appending the fragment once avoids triggering a reflow for every individual insertion.

- Batch reads then writes: read all needed measurements first, store them in variables, then apply all DOM writes afterward.
- DocumentFragment lets you build a batch of nodes off-DOM and insert them all in a single operation.
- requestAnimationFrame schedules visual updates in sync with the browser's paint cycle, avoiding redundant work.

Common Pitfalls

- Setting innerHTML with any string containing user-generated content, opening an XSS vulnerability.
- Attaching a separate event listener to every item in a large or dynamic list instead of using event delegation on a parent.
- Forgetting that querySelector/getElementById return null when nothing matches, then calling a method on that null and crashing.
- Iterating a live HTMLCollection (from getElementsByClassName/TagName) while adding or removing matching elements, producing unpredictable skipped/duplicated iterations.
- Interleaving DOM reads (offsetHeight, getBoundingClientRect) and writes (style changes) inside a loop, causing repeated forced layout recalculation.
- Attaching event listeners without ever removing them on cleanup, leaking memory in long-lived single-page applications.
- Using element.onclick = fn, which silently overwrites any previously assigned handler instead of adding an additional one.

Key Takeaways

- querySelector/querySelectorAll with standard CSS selectors should be your default for element selection.
- textContent is always safe; innerHTML with any untrusted string is an XSS risk.
- Event bubbling makes delegation possible — one listener on a stable parent can handle events for children that don't exist yet.
- closest(selector) is the standard tool for identifying which delegated child was actually interacted with.
- Batch DOM reads separately from DOM writes to avoid forced synchronous layout recalculation (layout thrashing).

Learn More

MDN — Introduction to the DOM

https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction

MDN — EventTarget.addEventListener()

<https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener>

web.dev — Avoid large, complex layouts and layout thrashing
<https://web.dev/articles/avoid-large-complex-layouts-and-layout-thrashing>